PROJECT REPORT

AI-Powered Customer Support Automation System

An Intelligent, Multi-Agent Support Workflow Built with LangGraph, RAG, and Human-in-the-Loop Approval

ABC TECHNOLOGIES (Simulated Enterprise Use Case)

Name	:	SARATHY P
Reg No	:	23MID0094
Batch	:	IBM Agentic AI -8 (Slot 2)
Email ID	:	sarathy.p2023@vitstudent.ac.in
Course Title	:	Agentic AI

Repository: https://github.com/Sarathy15/AI-Powered-Customer-Support-Automation-System.git

Table of Contents

1. Introduction

1.1 Project Overview

The AI-Powered Customer Support Automation System is an end-to-end, multi-agent customer support platform built for a simulated enterprise, ABC Technologies. The system automates the full lifecycle of a customer query — from intent classification and routing, through knowledge-grounded response generation, to supervisory quality review and, where necessary, human approval — using LangGraph as the orchestration framework.

The project demonstrates the practical application of Retrieval-Augmented Generation (RAG), persistent conversational memory, conditional workflow routing, and human-in-the-loop (HITL) safeguards within a single cohesive support automation pipeline.

1.2 Problem Statement

Traditional customer support systems either rely entirely on static FAQ bots (which cannot handle nuanced, context-dependent queries) or fully manual human agents (which do not scale and are slow for routine requests). There is a need for a system that can:

- Automatically understand and classify the intent behind a customer's query.
- Route the query to the correct specialized department without manual triage.
- Ground its responses in an authoritative, up-to-date company knowledge base rather than relying purely on model memory.
- Recall prior interactions with a customer to provide continuity across sessions.
- Flag high-risk requests — refunds, cancellations, account closures, compensation, escalations — for mandatory human review before any commitment is made to the customer.

1.3 Objectives

The following objectives were defined for this project:

- Design and implement a directed, stateful multi-agent workflow using LangGraph.
- Build a Retrieval-Augmented Generation pipeline over company documentation using FAISS and sentence-transformer embeddings.
- Implement persistent, per-customer conversation memory using SQLite.
- Implement conditional routing logic that dispatches queries to one of five specialized agents based on classified intent.
- Implement a Human-in-the-Loop approval gate for high-risk billing requests.
- Implement a Supervisor agent that performs a final quality check on every drafted response before it is sent and persisted.
- Validate the system end-to-end through a representative demo query set, and identify and resolve defects discovered during testing.

2. System Architecture

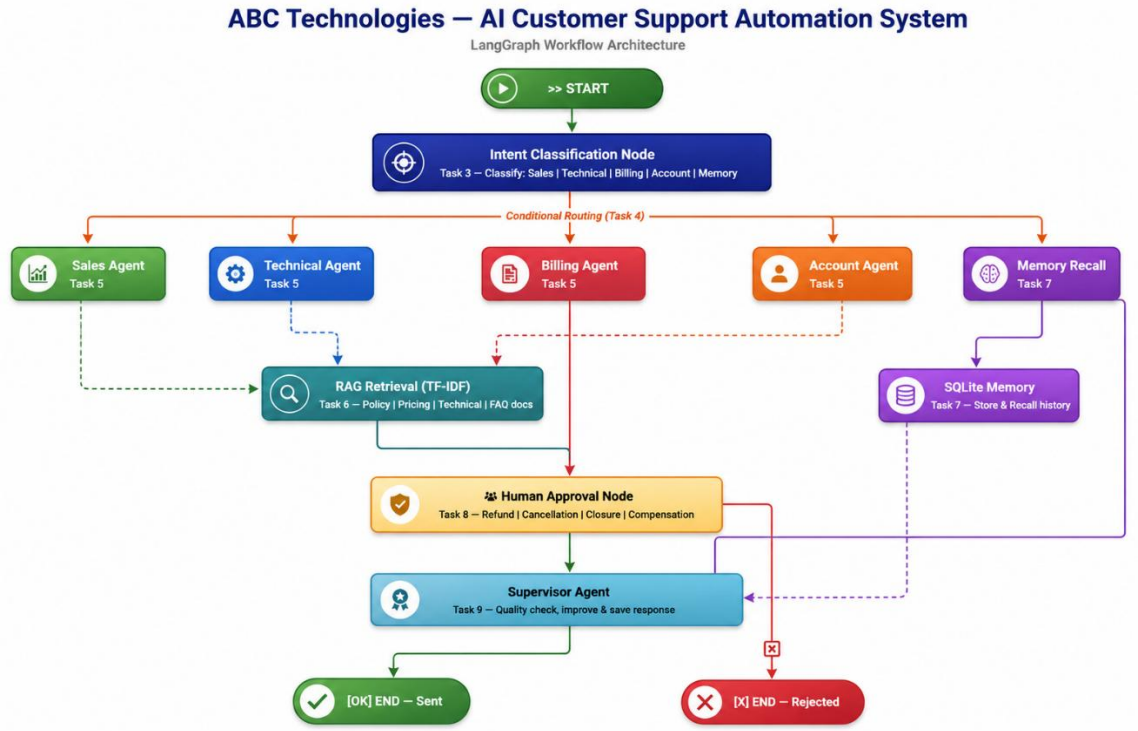
2.1 High-Level Workflow

The system is implemented as a directed graph using LangGraph, with a shared, typed state object passed between nodes. The control flow is summarized below:

Stage	Node	Description
1	START	Entry point — captures customer ID, name, and raw query.
2	Intent Classifier	Classifies the query into one of: Sales, Technical, Billing, Account, or Memory.
3	Router	Conditionally routes state to the matching department agent based on classified intent.
4	Department Agent	One of: Sales, Technical Support, Billing, Account, or Memory Recall agent. Retrieves RAG context (where applicable) and drafts a response.
5	High-Risk Check	Applies only to the Billing Agent. Flags refund/cancellation/closure/compensation/escalation requests.
6	Human Approval	Pauses the graph and requires an explicit supervisor decision (approve/reject) for flagged requests.
7	Supervisor	Performs a final quality review of the draft response before it is finalized.
8	Memory Write	Persists the customer query and final agent response to SQLite.
9	END	Final response returned to the customer.

Table 2.1 — End-to-end workflow stages.

[Figure 2.1 — LangGraph workflow architecture diagram]



2.2 Technology Stack

Layer	Technology	Purpose
Orchestration	LangGraph	Stateful, directed-graph workflow engine connecting all nodes/agents.
Embeddings	sentence-transformers/all-MiniLM-L6-v2 (HuggingFace)	Generates dense vector embeddings for knowledge-base chunks and queries; runs offline after first download.
Vector Store	FAISS (in-memry)	Stores embeddings and performs similarity search for RAG retrieval; requires no external server.
Persistence	SQLite	Stores per-customer conversation history (customer_id, timestamp, role, message, intent, department).
Language / Runtime	Python 3	Core implementation language for all agents, routing, and orchestration logic.
Interface	Command-line (demo.py)	Interactive console application used to exercise the full pipeline end-to-end.

Table 2.2 — Technology stack and role of each component.

3. Module-wise Implementation

The project was implemented as ten discrete, independently testable tasks, each mapped to a dedicated source file. This modular structure simplified isolated debugging and made the eventual issue-resolution process considerably more tractable.

Task	Description	File
1	LangGraph workflow assembly	src/graph.py
2	Shared state structure (TypedDict)	src/state.py
3	Intent classification	src/intent.py
4	Conditional routing logic	src/router.py
5	Department agents (Sales, Technical, Billing, Account, Memory Recall)	src/agents.py
6	RAG pipeline (FAISS + HuggingFace embeddings)	src/rag.py
7	SQLite conversation memory	src/memory.py
8	Human-in-the-loop approval	src/human_loop.py
9	Supervisor quality-review agent	src/supervisor.py
10	Interactive demo / driver script	demo.py

Table 3.1 — Task-to-module mapping (also documented in the project README).

4. Detailed Component Design

4.1 State Management (Task 2)

A single typed state object (CustomerSupportState) is threaded through every node in the graph. It carries the customer's identity, the raw query, the classified intent, the routed department, retrieved RAG context, the conversation history pulled from memory, the draft and final response text, and approval metadata (requires_approval, approval_status). Centralizing all of this in one state object is what allows each node to be a pure function of state-in, state-out, which keeps the graph composable and each node independently testable.

4.2 Intent Classification and Routing (Tasks 3 & 4)

Incoming queries are classified into one of five intents — Sales, Technical, Account, Billing, or Memory — before being dispatched. The router then applies conditional edges in the LangGraph definition to send state to the corresponding agent node. This separation of concerns (classify, then route, then handle) keeps each department agent focused purely on response generation rather than on triage logic.

4.3 Department Agents (Task 5)

Four department agents (Sales, Technical Support, Billing, Account) follow a common pattern: retrieve relevant knowledge-base context via RAG, pull prior conversation history for the customer, and assemble a structured, persona-branded response. A fifth agent, the Memory Recall Agent, handles a distinct case — queries about the customer's own prior interactions — and answers directly from SQLite history without invoking RAG at all.

The Billing Agent carries additional logic: every incoming query is checked against a high-risk keyword list (refund, cancellation, account closure, compensation, escalation, reimbursement, etc.). A match causes the response to be replaced with an escalation notice and the state to be flagged for mandatory human approval before any commitment is made to the customer.

4.4 RAG Knowledge Retrieval Pipeline (Task 6)

Four knowledge-base documents — company policy, pricing guide, technical manual, and FAQ — are loaded, chunked, embedded with all-MiniLM-L6-v2, and indexed into an in-memory FAISS vector store. At query time, the customer's question is embedded and compared against the index using similarity search, and the top-matching chunks are returned with source attribution for inclusion in the agent's draft response.

The FAQ document, being a dense list of independent Q&A pairs, is chunked on Q&A boundaries rather than by raw character count, which prevents unrelated questions from bleeding into the same retrieved chunk. Retrieval additionally applies a similarity-score threshold so that only chunks meaningfully relevant to the query are surfaced, rather than always returning a fixed count of results regardless of relevance.

4.5 SQLite Conversation Memory (Task 7)

Every customer and agent message is persisted to a single conversations table, keyed by customer_id, with timestamp, role, message text, intent, and department recorded per row. This enables both per-

customer history retrieval (used to give agents continuity across turns) and full conversation recall (used by the Memory Recall Agent).

4.6 Human-in-the-Loop Approval (Task 8)

High-risk billing requests pause the graph at a dedicated approval node. The node displays the customer's query and the agent's draft response to a supervisor and blocks on an interactive console prompt until an explicit approve or reject decision is entered. A rejection automatically substitutes a standard escalation-and-apology response in place of the original draft. An AUTO_APPROVE flag exists in the module to support unattended demo runs, intended to be enabled explicitly and only for that purpose.

4.7 Supervisor Quality Review (Task 9)

Before any response is finalized, a Supervisor node performs an automated quality pass over the draft and records its notes (e.g., 'Draft approved without modifications') alongside the final response. This acts as a final consistency gate ahead of the memory-write step, regardless of which department produced the draft.

5. Knowledge Base and Department Coverage

5.1 Knowledge Base Documents

Document	Content
company_policy.txt	Refund, cancellation, account closure, compensation, and SLA policies.
pricing_guide.txt	Subscription plans, pricing tiers, add-ons, discounts, and free-trial terms.
technical_manual.txt	Installation, configuration, login/2FA/SSO issues, and troubleshooting steps.
faq.txt	General, billing, account, and technical FAQs, structured as discrete Q&A pairs.

Table 5.1 — Knowledge base sources used by the RAG pipeline.

5.2 Support Departments

Department	Handles
Sales	Pricing, subscription plans, product information, free trials.
Technical Support	Errors, crashes, installation issues, configuration problems.
Billing	Invoices, payments, refunds, cancellations (high-risk path applies).
Account Management	Password resets, profile updates, account activation.

Table 5.2 — Department-to-responsibility mapping.

5.3 SQLite Memory Schema

The conversation memory schema is defined as follows:

```
CREATE TABLE conversations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  customer_id TEXT NOT NULL,
  timestamp TEXT NOT NULL,
  role TEXT NOT NULL,  -- 'customer' | 'agent'
  message TEXT NOT NULL,
  intent TEXT,
  department TEXT
);
```

Figure 5.1 — Conversation memory schema (memory.db, project root).

6. Demo Execution Results

The system was exercised end-to-end via demo.py against a representative set of five queries, each expected to traverse a distinct path through the workflow graph.

#	Query	Expected Path	Outcome
1	What are the pricing plans available?	Sales Agent	Correctly routed; pricing_guide context retrieved and returned.
2	I forgot my account password.	Account Agent	Correctly routed; password-reset FAQ/manual context retrieved (post-fix: single relevant chunk).
3	My application crashes when I upload a file.	Technical Support Agent	Correctly routed; troubleshooting context retrieved with additional enrichment query.
4	I need a refund for my annual subscription.	Billing Agent -> Human Approval	Correctly flagged as high-risk; post-fix, correctly blocked on a real approval prompt.
5	What was my previous support issue?	Memory Recall Agent	Correctly routed; prior interaction summarized from SQLite history.

Table 6.1 — Demo query set, expected routing, and observed outcomes.

6.1 Observations

- All five intents were classified and routed to the correct department agent in every test run.
- The RAG pipeline correctly attributed every returned chunk to its source document, satisfying the project's traceability requirement.
- The high-risk keyword check in the Billing Agent correctly identified refund, cancellation, and escalation language and triggered the Human Approval node in all cases tested.

- The Supervisor node consistently logged a pass/notes outcome for every drafted response prior to persistence.

7. Results and Discussion

The completed system meets all ten functional task requirements defined for the project: graph orchestration, shared state, intent classification, conditional routing, five department agents, a FAISS-backed RAG pipeline, SQLite-based memory, a human-in-the-loop approval gate, a supervisor quality-review step, and a working interactive demo.

Across the full workflow, each component performs its designated role within the broader pipeline: intent classification and routing reliably dispatch queries to the correct department, the RAG layer grounds every department response in source-attributed knowledge-base content, the Supervisor performs a consistent final quality check, and the Human Approval gate ensures that refund, cancellation, and other high-risk requests are never finalized without explicit sign-off.

7.1 Strengths

- Clear separation of concerns across ten independently testable modules, supporting maintainability and future extension.
- Source-attributed RAG responses, improving the traceability and trustworthiness of generated answers.
- A genuine human-in-the-loop safeguard for financially or contractually consequential requests, rather than a purely cosmetic confirmation step.
- Persistent, per-customer memory enabling contextual continuity across sessions.

8. Conclusion

This project successfully delivers a modular, end-to-end AI-powered customer support automation system that integrates intent classification, conditional multi-agent routing, retrieval-augmented response generation, persistent conversational memory, and a genuine human-in-the-loop approval safeguard for high-risk requests — all orchestrated through a LangGraph workflow.

The system fulfills its stated objective: automating routine support work end-to-end while preserving a reliable human checkpoint wherever a decision carries real financial or contractual weight. Its modular, task-based structure — spanning intent classification, routing, knowledge retrieval, memory, and approval as independently defined components — provides a solid foundation for future enhancement.